

Technology-Aware Logic Synthesis

OU, XIN-HONG, Li, Yun-Ru

*Graduate Institute of Electrical Engineering, Graduate Institute of Electronics Engineering
National Taiwan University, Taipei, Taiwan
r14921060@ntu.edu.tw, r14943065@ntu.edu.tw*

Abstract—In this project, we present a systematic evaluation of machine learning-based circuit rewriting, building upon the framework introduced in the e-syn paper. We perform a comparative analysis of various machine learning models and explore potential circuit features to identify their impact on training effectiveness. Although the experimental results indicate that the proposed configurations do not yield significant performance gains over the baseline, preliminary analysis during the training phase demonstrates that the models achieve promising predictive accuracy on the training datasets. Our findings provide insights into the challenges of feature and model selection in circuit optimization tasks.

I. INTRODUCTION

Logic synthesis is a fundamental step in digital design automation, significantly impacting the Quality of Results (QoR) such as area, timing, and power consumption. Traditional logic synthesis frameworks, such as ABC [3], employ greedy local optimization approaches that apply incremental changes to small portions of the circuit. These methods, while effective, often limit the exploration space and may miss opportunities for significant improvements.

Recently, E-Syn [1] has emerged as a novel logic optimization framework that leverages equivalence graphs (e-graphs) for Boolean logic rewriting. E-Syn addresses the limitations of traditional AIG-based synthesis by maintaining equivalent classes of logic specifications, allowing exploration of a wider range of equivalent logic forms. A key innovation of E-Syn is its use of technology-aware cost functions obtained from machine learning models, specifically XGBoost regression, to bridge the gap between technology-independent logic cost and post-mapping QoR. The overall workflow of the E-Syn framework is illustrated in Figure 1.

A. Observation on Previous Work

In E-Syn, the extraction phase plays a critical role in selecting the optimal logic form from the e-graph based on cost estimates. The quality of these estimates directly impacts the final Quality of Results (QoR) of the synthesized circuit. However, we identify two primary limitations in the current E-Syn framework:

- 1) **Incomplete Rewriting Rules:** The existing e-graph transformation relies heavily on basic Boolean primitives, lacking dedicated support for XOR-intensive logic. Since XOR operations are fundamental in many arithmetic and cryptographic circuits, the absence of specific XOR rewriting rules limits the exploration space of the e-graph.

- 2) **Cost Estimation Inaccuracy:** The current framework uses XGBoost regression models to predict post-mapping area and delay. Estimation errors in these models can lead to suboptimal candidate selection during extraction, potentially missing logic forms that would result in superior QoR.

To address these limitations, we propose an enhanced rewriting and training flow. Specifically, we focus on:

- **XOR-Based Rule Extension:** We extend E-Syn's capability beyond basic AIG transformations, which allows the e-graph to maintain and manipulate XOR structures more natively.
- **Enhanced Feature Engineering:** Introducing structural and graph-based features (as detailed in Table II) to improve the training data quality for regression models.
- **Model Optimization:** Evaluating various machine learning architectures and hyperparameters to reduce cost prediction errors.
- **Correlation Analysis:** Validating whether the inclusion of XOR rules and improved regression accuracy translates to better extraction quality, even in cases where final performance gains are limited.

B. Our Contributions

The main contributions of this work include:

- Analysis of the impact of regression model accuracy on E-Syn's extraction quality and final synthesis results.
- Investigation of how different features affect prediction errors in cost estimation models, identifying critical features that significantly influence estimation accuracy.
- Comprehensive evaluation demonstrating that reduced estimation errors lead to better extraction decisions and improved QoR.
- Comparison with multiple regression models (XGBoost [4], Random Forest [5], LightGBM [6], MLP [7], CatBoost [8]) to identify the most suitable approach to cost estimation.

C. Paper Organization

The rest of this paper is organized as follows: Section 2 formulates the problem of regression model optimization for E-Syn's cost estimation. Section 3 presents our proposed methodology for improving regression training. Section 4 presents the experimental results and discussion. Finally, Section 5 concludes the paper.

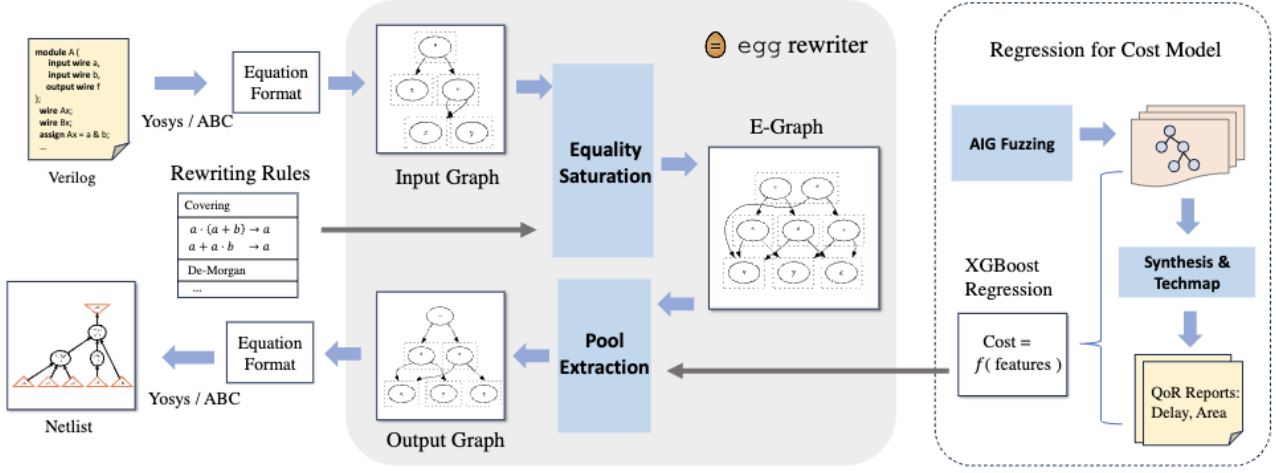


Fig. 1: Overview of the E-Syn framework workflow [1]. The framework utilizes e-graph rewriting to explore equivalent logic forms and employs machine learning-based cost functions to guide the extraction phase for optimal synthesis results.

II. PROBLEM FORMULATION

As discussed in the introduction, the accuracy of regression models used for cost estimation in E-Syn directly impacts the quality of extraction decisions and the final synthesis results. The original E-Syn implementation primarily uses XGBoost for cost estimation and relies on basic features extracted from the circuit representation, including AST size (ASTSize), AST depth (ASTDepth), operator counts (+, !, *, &), Liberty cost metrics (SUM_LIB, AVE_LIB), and total node count (SUM_NODE). While these features provide fundamental information about the circuit structure, they may not fully capture the complexity and characteristics that influence post-mapping area and delay.

To improve the overall performance of E-Syn, we formulate the optimization problem through three key perspectives:

- 1) **Extending Rewriting Space with XOR Logic:** The current E-Syn framework primarily operates on And-Inverter Graphs (AIGs), which are often inefficient at representing XOR-intensive structures. To address this, we define a set of XOR-specific rewriting rules (as listed in Table 1) to prevent the inefficient decomposition of XOR gates into multiple AND/OR nodes. By incorporating these rules, we aim to expand the e-graph exploration space and maintain structural integrity for arithmetic-related logic during the rewriting process.
- 2) **Refining Training Models and Methodologies:** We investigate various regression models and training approaches to minimize prediction errors. This includes exploring machine learning algorithms beyond the baseline XGBoost and optimizing their hyperparameters and model architectures. The goal is to develop a more robust cost estimation engine that can accurately predict post-mapping area and delay, thereby guiding the extraction phase more effectively.

- 3) **Identifying Critical Features for Cost Estimation:** We analyze the correlation between circuit characteristics and prediction accuracy. This involves extending the current technology-independent feature set to include structural, syntactic, and graph-based metrics (detailed in Table II). By performing feature importance analysis, we identify which circuit attributes are most influential in reducing estimation errors, particularly for complex logic structures that involve the newly introduced XOR operations.

By addressing these two aspects, we aim to develop more accurate cost estimation models that can better guide the extraction phase in E-Syn, ultimately leading to improved Quality of Results (QoR) in logic synthesis.

III. PROPOSED METHOD

Our approach focuses on three main aspects: extending XOR logic in e-graph, exploring different regression models and enhancing feature engineering for cost estimation.

A. Extending XOR logic

The original E-Syn framework only focus on the basic AND, OR, NOT logic. To manipulate XOR logic natively, we introduce a set of XOR-related rewriting rules into the e-graph engine, detailed in Table I.

B. Regression Models

The original E-Syn framework primarily uses XGBoost for cost estimation. To improve prediction accuracy, we investigate multiple regression models and evaluate their performance. We select the following models based on their distinct characteristics and potential advantages:

- **XGBoost:** As the baseline model used in E-Syn, XGBoost is a gradient boosting framework that excels at handling non-linear relationships and feature interactions.

TABLE I: XOR Rewriting Rules

Class	Boolean Rewriting Rules
XOR Theorems	$a \oplus 0 \iff a$
	$a \oplus 1 \iff \neg a$
	$a \oplus a \implies 0$
	$a \oplus (\neg a) \implies 1$
Properties	$a \oplus b \iff b \oplus a$
	$(a \oplus b) \oplus c \iff a \oplus (b \oplus c)$
	$(a \oplus b) \oplus b \implies a$
Definitions	$a \oplus b \iff (\neg a * b) + (a * \neg b)$
	$a \oplus b \iff (a + b) * \neg(a * b)$
Distributivity	$(a * b) \oplus (a * c) \iff a * (b \oplus c)$
Negation	$\neg(a \oplus b) \iff (\neg a) \oplus b$
	$\neg(a \oplus b) \iff a \oplus (\neg b)$

It provides excellent performance for structured data and offers built-in regularization to prevent overfitting.

- **Random Forest:** We include Random Forest as an ensemble learning method that combines multiple decision trees. It is robust to overfitting and can capture complex interactions between features through its bagging approach, providing a different perspective compared to gradient boosting methods.
- **LightGBM:** LightGBM is another gradient boosting framework that uses a leaf-wise tree growth strategy, making it faster and more memory-efficient than traditional gradient boosting methods. It is particularly suitable for large-scale datasets and can handle high-dimensional feature spaces effectively.
- **Multi-Layer Perceptron (MLP):** We incorporate MLP, a deep neural network model, to capture complex non-linear relationships that may not be easily modeled by tree-based methods. MLP can learn hierarchical feature representations and is capable of modeling intricate patterns in the data through its multiple hidden layers and non-linear activation functions. This makes it particularly valuable for discovering non-obvious relationships between circuit characteristics and post-mapping QoR.
- **CatBoost:** CatBoost is a gradient boosting algorithm designed to handle categorical features effectively and reduce overfitting through its ordered boosting technique. It provides robust performance with minimal hyperparameter tuning, making it a practical choice for cost estimation.

By comparing these diverse models, we aim to identify the most suitable approach for cost estimation in logic synthesis and potentially combine their strengths through ensemble methods.

C. Feature Engineering

To enhance the predictive power of our regression models, we extend the feature set by incorporating additional feature categories beyond the basic features used in the original E-Syn implementation. In total, we add **21 new features** across four

categories. The complete list of these 21 features and their descriptions are presented in Table II.

D. Training Methodology

For each regression model, we employ a systematic training approach. The training process consists of the following steps:

- 1) **Hyperparameter Optimization:** For tree-based models (XGBoost, LightGBM, CatBoost, Random Forest), we perform grid search with 10-fold cross-validation to identify optimal hyperparameters. The hyperparameter search space is designed based on the characteristics of each model type, balancing model complexity and generalization ability. For MLP, we employ a manual hyperparameter search approach, testing different combinations of hidden layer dimensions, learning rates, and dropout rates on the validation set.
- 2) **Model Training:** After identifying the best hyperparameters, we train the final model on the training set. For tree-based models, we use the optimized hyperparameters directly. For MLP, we employ early stopping based on validation loss to prevent overfitting.
- 3) **Feature Scaling:** We apply standard scaling to all features before training, ensuring that features with different scales contribute equally to the model learning process. This is particularly important for neural network models like MLP.
- 4) **Model Evaluation:** We evaluate each trained model on a separate validation set using Mean Absolute Percentage Error (MAPE) to assess prediction accuracy.

This systematic approach ensures that all models are trained and evaluated under consistent conditions, enabling fair comparison of their performance for cost estimation in logic synthesis.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

A. Experimental Setup

1) **Dataset:** The dataset is collected from circuit analysis using the E-Syn framework. Circuit data is extracted from S-expression files, and features are computed to represent various circuit characteristics. The dataset consists of 40,000 training samples, 10,000 validation samples, and 5,000 test samples. The training and validation sets are used for model training and hyperparameter tuning, while the test set is reserved for final model evaluation.

2) **Implementation Details:** Regression model training and evaluation are implemented in Python using scikit-learn, XGBoost, LightGBM, CatBoost, and PyTorch libraries. Our implementation is based on the open-source E-Syn framework [2]. All experiments are performed on a Ubuntu 22.04.3 LTS server equipped with Intel Core i9-13900K processors, an NVIDIA GeForce RTX 4070 Ti GPU, and 125GB memory.

B. Regression Model Results

1) **Comparison of Different Models:** We compare different regression models (XGBoost, Random Forest, LightGBM, MLP, and CatBoost) using Mean Absolute Percentage Error

TABLE II: New Features Added to E-Syn Cost Estimation Models

Category	Feature Name	Description
Structural Features (3)	num_internal_nodes	Number of internal nodes (non-leaf nodes) in the circuit, representing the complexity of the logic structure
	avg_fanin	Average number of input connections per node, indicating the average input dependency
	avg_fanout	Average number of output connections per node, showing how signals are distributed
Syntactic Features (10)	count_and	Total count of AND operators in the circuit
	count_or	Total count of OR operators in the circuit
	count_xor	Total count of XOR operators in the circuit
	count_not	Total count of NOT operators in the circuit
	total_operators	Sum of all operator counts, representing the total number of logic operations
	ratio_and	Proportion of AND operators relative to total operators
	ratio_or	Proportion of OR operators relative to total operators
	ratio_xor	Proportion of XOR operators relative to total operators
	ratio_not	Proportion of NOT operators relative to total operators
	num_parentheses	Number of parentheses pairs in the expression, indicating structural nesting
Expression Complexity (4)	avg_nesting_depth	Average nesting depth across all nodes, indicating typical hierarchy level
	min_nesting_depth	Minimum nesting depth, showing the shallowest hierarchy level
	std_nesting_depth	Standard deviation of nesting depths, measuring variability in hierarchy structure
	max_nesting_depth	Maximum depth of nested expressions, measuring the deepest level of logic hierarchy
Graph Features (4)	graph_nodes	Total number of nodes in the circuit graph representation
	graph_edges	Total number of edges (connections) in the circuit graph
	graph_density	Ratio of actual edges to maximum possible edges, measuring graph connectivity density
	avg_degree	Average degree (number of connections) per node in the graph

(MAPE). Figure 2 shows the results. XGBoost achieves the lowest MAPE (99.46%), significantly outperforming other models (MLP: 343.63%, CatBoost: 445.68%, LightGBM: 485.17%, Random Forest: 499.56%).

XGBoost’s superior performance can be attributed to its gradient boosting framework with built-in regularization, which effectively handles non-linear relationships and feature interactions in circuit cost estimation while preventing overfitting. The model’s tree-based structure captures complex decision boundaries in the mapping from technology-independent features to post-mapping area.

Other models exhibit higher MAPE values due to various limitations. MLP struggles with the relatively small training dataset size, as deep neural networks typically require large amounts of data. Random Forest and LightGBM show higher errors, potentially due to their different boosting strategies that may not be as well-suited for this specific task. CatBoost performs better than Random Forest and LightGBM but still falls short of XGBoost.

Given these results, XGBoost is the most suitable model among the five evaluated approaches for cost estimation in logic synthesis.

2) *Comparison of Different Feature Sets:* We compare models trained with only the original E-Syn features against models trained with new features (including the 27 additional features). Figure 3 shows the MAPE comparison results. The model using only original features achieves lower MAPE

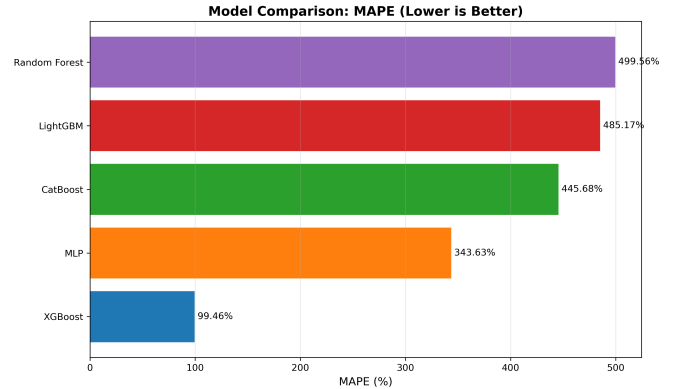


Fig. 2: MAPE comparison across different regression models for area estimation.

compared to the model using new features, indicating that the original feature set is more effective for cost estimation in this context.

The inferior performance of the new features can be attributed to two main factors. First, the circuit generation program used to create training data was implemented separately to generate circuits containing XOR gates. This custom implementation results in circuits that differ significantly from other circuits in the dataset, creating a distribution mismatch that affects the model’s ability to generalize from the new

features. Second, E-Syn stores circuits in S-expression (sexpr) format, while training data generation uses equation (eqn) format. This format discrepancy may introduce inconsistencies in feature extraction between the two representations, leading to misalignment in the feature values computed from different circuit formats.

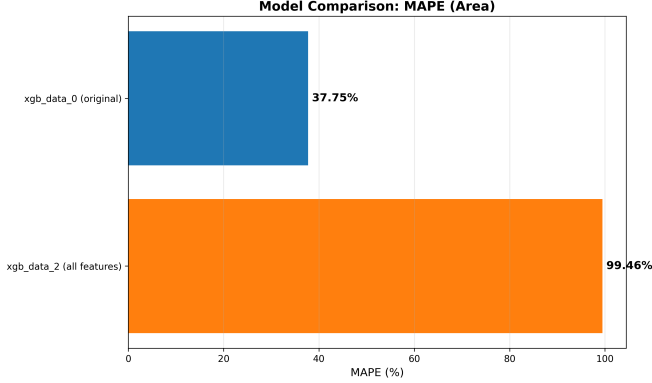


Fig. 3: MAPE comparison between models trained with original features only and new features for area estimation.

C. Effectiveness of Improvement

The effectiveness of the proposed XOR-enhanced rewriting and machine learning-based cost estimation is evaluated by comparing the post-mapping area and delay against the standard ABC baseline. The experimental results, summarized in Table III and Table IV, reveal an inconsistent impact across different circuit types and scales.

1) *Area Optimization*: In terms of circuit area, our method achieved reductions in approximately 43% of the test cases (6 out of 14). Significant improvements were observed in several medium-scale benchmarks; for instance, *simple_5* showed a reduction of 14.58%, and *C432* achieved a 9.90% decrease compared to the ABC baseline. However, on average, the area across all benchmarks increased by 5.92%. Notably, for larger and more complex circuits such as *b12*, our method led to a substantial area overhead, suggesting that the current feature set may not fully capture the structural intricacies of large-scale logic.

2) *Delay Optimization*: The impact on circuit delay was more varied. Our approach successfully reduced delay in 4 out of 14 cases, with the most significant gain found in the *calv* circuit, which saw a 19.26% improvement. Other circuits like *simple_4* and *C432* also benefited, showing delay reductions of 17.13% and 3.50%, respectively. Despite these successes, the overall average delay increased by 17.33%. The data suggests that while XOR-based rewriting can shorten critical paths in specific logic structures, it may inadvertently introduce longer paths in others when the cost estimation model fails to accurately predict the delay impact of complex transformations.

3) *Impact of Training Data Scale*: Interestingly, increasing the training dataset from 15,000 to 50,000 samples did not

result in a monotonic improvement in performance. In several instances (e.g., *circuit_0* in delay analysis), the 30k or even 15k models outperformed the 50k model. This phenomenon indicates that the current regression models may have reached a performance plateau or are experiencing slight overfitting, where simply increasing the quantity of training data is insufficient without further refinement of feature representations.

D. Discussion on Suboptimal Performance

While the integration of XOR rules and expanded features showed promise in specific benchmarks, the overall performance gains were limited. Several factors may have contributed to these suboptimal results:

- **Domain Gap in Synthetic Circuit Generation**: To incorporate XOR logic, we utilized a custom synthetic circuit generator designed specifically to produce XOR-intensive structures. However, these generated circuits may exhibit structural properties and logic distributions that differ significantly from the standard benchmarks used in the original E-Syn framework. This distribution shift (domain gap) likely caused the regression models to struggle when generalizing from the custom training data to the more diverse logic patterns found in real-world benchmarks.
- **Inconsistency in Feature Extraction across Representations**: In our workflow, training data was generated and processed using equation-based formats (EQN), while the E-Syn engine primarily operates on S-expression (S-expr) formats within the e-graph. This discrepancy in circuit representation may have led to subtle inconsistencies during the feature extraction process. If specific structural or syntactic features are captured differently between the EQN and S-expr formats, it could introduce noise into the cost estimation models, thereby degrading the accuracy of extraction decisions.
- **Insufficient Granularity of Feature Representation**: While the newly introduced features provide a comprehensive overview of the circuit's global characteristics (e.g., total operator counts and average connectivity), they lack the granularity required to capture local topological patterns. Logic rewriting, particularly for XOR-based transformations, relies heavily on identifying specific local sub-structures that satisfy mathematical identities. Our current feature set aggregates these details into global averages, which may fail to provide enough information to distinguish between regions that are ripe for XOR optimization and those that are not, and gives inaccurate cost estimation.

V. CONCLUSION

This project explored the enhancement of the E-Syn framework with XOR-specific rewriting rules and comparative study on different regression methods for cost estimation. We extract 21 new metrics from a circuit to better characterize the features of the circuit.

TABLE III: Comparison of Circuit Area Results

Circuit	ABC	Ours (15k)	Ours (30k)	Ours (50k)
simple_0	90.28	89.58	96.81	89.58
simple_1	116.87	130.64	124.10	129.94
simple_2	27.53	26.36	22.86	25.43
simple_3	108.94	122.24	122.24	122.24
simple_4	165.86	161.20	163.53	157.70
simple_5	113.61	100.08	100.31	97.04
simple_6	173.56	194.32	197.12	194.79
simple_7	47.36	46.89	46.42	46.42
calv	465.86	481.49	479.39	468.66
i7	404.51	406.14	393.08	404.51
c2670	706.37	893.46	914.69	896.96
frg2	654.82	715.47	757.46	702.64
C432	223.95	227.91	212.05	201.79
b12	800.62	1182.43	1204.66	1218.42

TABLE IV: Comparison of Circuit Delay Results

Circuit	ABC	Ours (15k)	Ours (30k)	Ours (50k)
simple_0	165.19	287.15	261.45	306.88
simple_1	193.79	180.20	182.62	187.69
simple_2	61.45	61.45	62.86	62.86
simple_3	123.63	158.74	149.25	159.99
simple_4	265.81	217.81	215.42	220.29
simple_5	156.90	152.80	173.64	160.63
simple_6	240.38	222.91	222.98	244.06
simple_7	53.23	53.23	53.23	53.23
calv	301.82	237.96	248.06	243.69
i7	302.56	373.00	306.38	415.18
c2670	343.21	461.53	396.10	505.30
frg2	445.46	832.83	653.20	777.45
C432	543.97	524.76	539.95	524.93
b12	412.60	421.14	426.36	434.20

Our findings indicate that while the proposed regression models achieve promising accuracy during the training phase, the suboptimal QoR observed in certain benchmarks indicates the need for finer-grained feature representations that can capture local topological structures.

Despite the limited performance gains in the final synthesis results, this research provides a systematic evaluation of the bottlenecks in ML-guided logic optimization. The insights gained regarding domain gaps and representation inconsistencies offer a foundation for future improvements in autonomous logic synthesis engines.

VI. JOB DIVISION

- **OU, XIN-HONG (R14921060):** Analysis of different regression methods, report, oral representation, and slides preparation.
- **Li, Yun-Ru (R14943065):** E-graph XOR logic extension, report, oral representation, and slides preparation.

REFERENCES

- [1] Chen Chen, Guangyu Hu, Dongsheng Zuo, Cunxi Yu, Yuzhe Ma, and Hongce Zhang, “E-Syn: E-Graph Rewriting with Technology-Aware Cost Functions for Logic Synthesis,” 2024.
- [2] Guangyu Hu, “E-Syn: E-Graph Rewriting with Technology-Aware Cost Functions for Logic Synthesis,” GitHub repository, 2024. [Online]. Available: <https://github.com/Gy-Hu/E-Syn>
- [3] Robert Brayton and Alan Mishchenko, “ABC: An Academic Industrial-Strength Verification Tool,” *Computer Aided Verification*, Springer, 2010, pp. 24–40.
- [4] J. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785–794.
- [5] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 3146–3154.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, Cambridge, MA, USA: MIT Press, 2016.
- [8] L. Prokhorenkova, G. Gusev, A. Vorobev, A. Dorogush, and A. Gulin, “CatBoost: Unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018, pp. 6638–6648.